

## OPEN TECH NOTES

*loạt tài liệu kỹ thuật miễn phí cho cộng đồng*

# ĐÁNH GIÁ MÔ HÌNH

**Biết mình đang cải thiện, hay chỉ đang đoán**

*Các phương pháp đánh giá, xây dựng bộ dữ liệu, và cạm bẫy thường gặp*

Phần 5 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

## Mục lục

|                                                                       |   |
|-----------------------------------------------------------------------|---|
| <b>Mở đầu</b> .....                                                   | 3 |
| <b>1. Vì sao đánh giá mô hình khó hơn phần mềm truyền thống</b> ..... | 3 |
| <b>2. Ba cách đánh giá phổ biến</b> .....                             | 4 |
| 2.1. Chỉ số tự động (metrics-based) .....                             | 4 |
| 2.2. LLM-as-judge .....                                               | 4 |
| 2.3. Đánh giá bởi con người .....                                     | 4 |
| <b>3. Xây dựng bộ dữ liệu đánh giá (eval set)</b> .....               | 5 |
| 3.1. Nguồn dữ liệu và tính đại diện .....                             | 5 |
| 3.2. Quy trình đánh giá liên tục .....                                | 6 |
| <b>4. Đánh giá theo từng thành phần hệ thống</b> .....                | 6 |
| <b>5. Rủi ro &amp; giới hạn thực tế</b> .....                         | 6 |
| 5.1. Data contamination và overfitting lên eval set .....             | 6 |
| 5.2. Thiên lệch của LLM-as-judge .....                                | 6 |
| <b>6. Checklist nhanh cho đội ngũ</b> .....                           | 7 |
| <b>Đọc thêm</b> .....                                                 | 7 |
| <b>Tài liệu tham khảo</b> .....                                       | 7 |

## Mở đầu

Bốn phần trước của loạt bài đã giới thiệu bốn công cụ để làm cho một foundation model làm đúng việc bạn cần: chọn mô hình nền, prompt engineering, RAG, và fine-tuning. Mỗi lần thử một thay đổi một prompt mới, một chiến lược truy xuất khác, một phiên bản fine-tune mới câu hỏi luôn quay lại: làm sao biết thay đổi đó thực sự tốt hơn, chứ không phải chỉ "trông có vẻ tốt hơn" qua vài ví dụ thử thủ công?

Đó chính là vai trò của việc đánh giá mô hình (evaluation) một cách có hệ thống xây dựng quy trình đo lường khách quan, lặp lại được, để so sánh các phiên bản khác nhau của hệ thống AI trước khi quyết định triển khai. Đây là chủ đề dễ bị đội kỹ sư bỏ qua nhất trong bốn tài liệu trước, không phải vì nó kém quan trọng, mà vì nó ít "nhìn thấy ngay kết quả" hơn nhưng lại là thứ quyết định liệu những cải tiến ở bốn phần trước có thực sự là cải tiến hay không. Tài liệu này trình bày vì sao đánh giá mô hình ngôn ngữ khó hơn kiểm thử phần mềm truyền thống, ba cách đánh giá phổ biến, cách xây dựng một bộ dữ liệu đánh giá đáng tin cậy, và những cạm bẫy thường gặp. Nội dung là bản tổng hợp gốc dựa trên kiến thức chung của ngành và các nguồn công khai, có trích dẫn ở cuối bài.

## 1. Vì sao đánh giá mô hình khó hơn phần mềm truyền thống

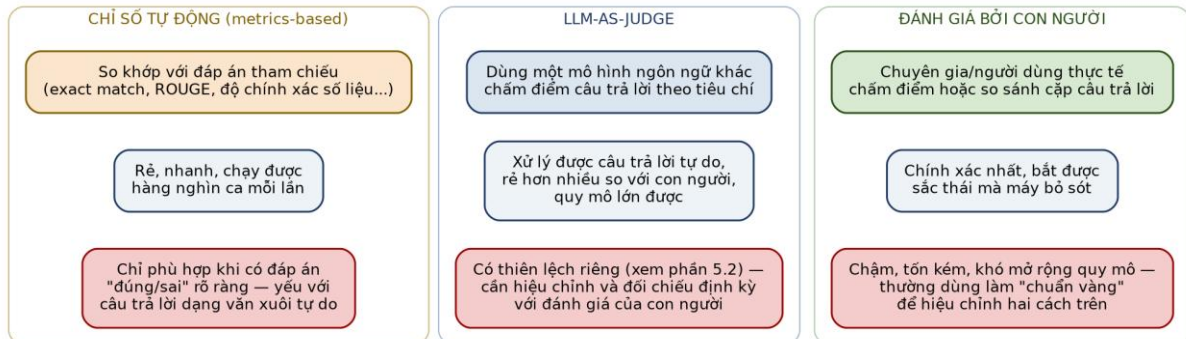
Kiểm thử phần mềm truyền thống thường có một đáp án đúng rõ ràng một hàm cộng hai số, hoặc một API trả về đúng định dạng, có thể kiểm tra bằng phép so sánh chính xác (assert). Đầu ra của một mô hình ngôn ngữ hiếm khi như vậy: cùng một câu hỏi có thể có nhiều câu trả lời đều đúng, diễn đạt khác nhau, độ dài khác nhau không có một chuỗi ký tự "đúng duy nhất" để so khớp.

- Không gian đầu ra gần như vô hạn: khác với phần mềm có đầu ra rời rạc, mô hình ngôn ngữ sinh văn bản tự do cùng một ý có hàng nghìn cách diễn đạt hợp lệ.
- Chất lượng mang tính chủ quan và đa chiều: một câu trả lời có thể đúng sự thật nhưng thiếu tự nhiên, hoặc đầy đủ nhưng quá dài dòng "tốt" phụ thuộc vào nhiều tiêu chí cùng lúc, không phải một trục đúng/sai.
- Hành vi không xác định (non-deterministic): cùng một đầu vào, mô hình có thể sinh ra các câu trả lời khác nhau ở những lần gọi khác nhau, khiến một phép kiểm thử chạy một lần không đủ tin cậy.

Vì những lý do này, một quy trình đánh giá nghiêm túc không thể chỉ dựa vào việc thử vài ví dụ và "nhìn thấy có vẻ ổn" cách làm đó dễ dẫn đến việc tối ưu nhầm theo cảm tính, đã được nhắc đến như một rủi ro ở tài liệu Prompt Engineering của loạt bài này.

## 2. Ba cách đánh giá phổ biến

Trong thực tế, ba cách tiếp cận sau thường được kết hợp với nhau, mỗi cách bù đắp điểm yếu của cách kia:



Hình 1. Ba cách đánh giá phổ biến chỉ số tự động, LLM-as-judge, và đánh giá bởi con người cùng đánh đổi giữa tốc độ, chi phí, và độ chính xác.

### 2.1. Chỉ số tự động (metrics-based)

Các chỉ số như exact match (khớp chính xác), BLEU và ROUGE (đo độ trùng lặp n-gram giữa câu trả lời và đáp án tham chiếu, ban đầu phát triển cho dịch máy và tóm tắt văn bản) cho phép chạy đánh giá cực nhanh và rẻ trên quy mô lớn. Điểm yếu là chúng đo sự trùng khớp về mặt câu chữ, không đo ý nghĩa một câu trả lời đúng ý nhưng diễn đạt khác có thể bị chấm điểm thấp một cách sai lệch. Nhóm chỉ số này phù hợp nhất khi tác vụ có đáp án khá cố định: trích xuất dữ liệu có cấu trúc, phân loại, hay tính toán số liệu.

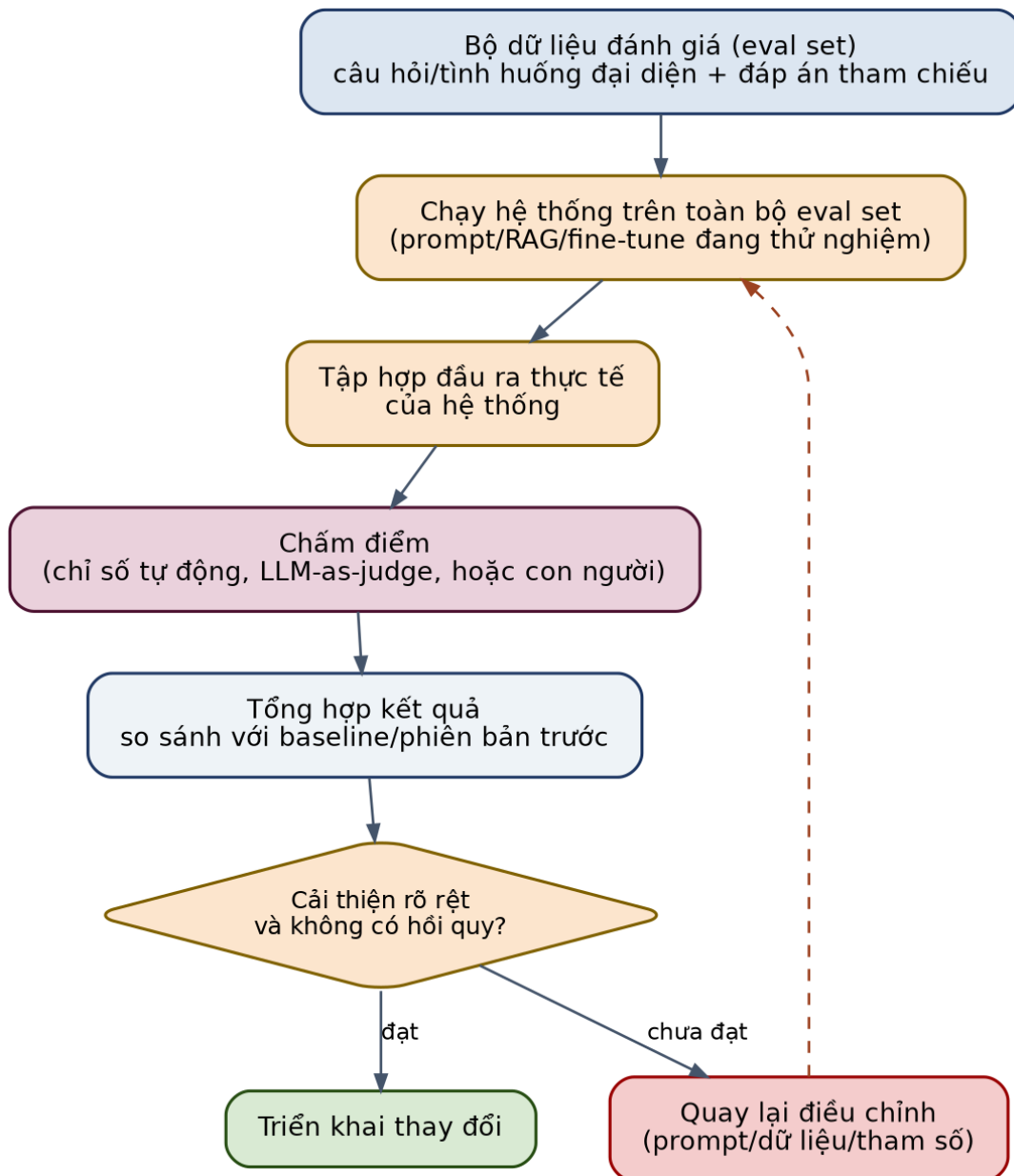
### 2.2. LLM-as-judge

LLM-as-judge dùng một mô hình ngôn ngữ khác (thường là một mô hình mạnh) để chấm điểm hoặc so sánh các câu trả lời theo một bộ tiêu chí mô tả bằng lời cách tiếp cận này được khảo sát và phổ biến rộng rãi qua nghiên cứu MT-Bench/Chatbot Arena (Zheng và cộng sự, 2023). Ưu điểm là xử lý được câu trả lời dạng văn xuôi tự do, chi phí thấp hơn nhiều so với thuê người đánh giá, và có thể chạy ở quy mô lớn gần như tự động hoàn toàn. Vì đây là công cụ nhanh và linh hoạt nhất trong ba cách, nó thường là lựa chọn mặc định cho việc đánh giá liên tục trong quá trình phát triển nhưng có những thiên lệch riêng cần lưu ý, sẽ bàn ở phần 5.2.

### 2.3. Đánh giá bởi con người

Chuyên gia trong lĩnh vực hoặc người dùng thực tế chấm điểm trực tiếp, hoặc so sánh cặp (A/B) giữa hai câu trả lời để chọn ra câu tốt hơn đây vẫn là cách chính xác nhất, bắt được những sắc thái tinh tế (giọng điệu, sự phù hợp văn hoá, cảm nhận tổng thể) mà cả chỉ số tự động lẫn LLM-as-judge đều có thể bỏ sót. Đánh đổi là chi phí, thời gian, và khó mở rộng quy mô vì vậy đánh giá bởi con người thường được dùng có chọn lọc, làm "chuẩn vàng" để hiệu chỉnh và kiểm chứng định kỳ cho hai cách còn lại, hơn là chạy trên mọi thay đổi nhỏ.

### 3. Xây dựng bộ dữ liệu đánh giá (eval set)



Hình 2. Quy trình đánh giá liên tục: chạy hệ thống trên eval set, chấm điểm, so sánh với phiên bản trước, và quyết định triển khai hay tiếp tục điều chỉnh.

#### 3.1. Nguồn dữ liệu và tính đại diện

Một bộ eval set tốt cần phản ánh đúng phân bố câu hỏi/tình huống thực tế mà hệ thống sẽ gặp trong sản xuất không chỉ những trường hợp "dễ" hay lý tưởng. Nguồn dữ liệu thường đến từ: log câu hỏi thực tế của người dùng (sau khi ẩn danh hoá thông tin nhạy cảm), các trường hợp lỗi đã phát hiện được trong quá khứ (regression set đảm bảo lỗi cũ không tái diễn), và các trường hợp biên (edge case) được đội kỹ sư chủ động nghĩ ra để kiểm tra giới hạn của hệ thống. Một eval set chỉ gồm các câu hỏi "dễ" sẽ cho điểm số đẹp nhưng không phản ánh đúng chất lượng thực tế.

## 3.2. Quy trình đánh giá liên tục

Đánh giá hiệu quả nhất khi được tích hợp vào quy trình phát triển như một bước bắt buộc, không phải một hoạt động rời rạc thỉnh thoảng mới làm: mỗi khi thay đổi prompt, cấu hình RAG, hay fine-tune một phiên bản mới, hệ thống chạy lại toàn bộ eval set, so sánh điểm số với phiên bản đang chạy production (baseline), và chỉ triển khai khi có cải thiện rõ rệt mà không gây hồi quy (regression) ở những trường hợp trước đó đã làm tốt. Cách làm này biến việc đánh giá từ một bước kiểm tra cuối cùng thành một vòng phản hồi liên tục trong suốt quá trình phát triển.

## 4. Đánh giá theo từng thành phần hệ thống

Một hệ thống AI production thường có nhiều thành phần ghép lại như đã thấy ở tài liệu RAG của loạt bài, một câu trả lời sai có thể do bước truy xuất lấy nhầm tài liệu, hoặc do mô hình sinh văn bản dùng sai ngữ cảnh dù truy xuất đúng. Đánh giá toàn bộ hệ thống như một hộp đen (chỉ nhìn câu trả lời cuối cùng) giúp biết kết quả tổng thể có tốt hay không, nhưng không cho biết nên sửa ở đâu khi kết quả chưa tốt.

Vì vậy, đánh giá hiệu quả thường tách theo từng lớp: với hệ thống RAG, đo riêng chất lượng truy xuất (precision/recall@k như đã nói ở tài liệu RAG) tách biệt với chất lượng câu trả lời cuối; với mô hình đã fine-tune, so sánh riêng với mô hình gốc trên cả tác vụ mục tiêu lẫn các khả năng chung khác (để phát hiện catastrophic forgetting như đã nói ở tài liệu Fine-tuning). Nguyên tắc chung: càng cô lập được thành phần gây lỗi, càng dễ và rẻ để sửa đúng chỗ, thay vì thử thay đổi ngẫu nhiên toàn bộ hệ thống rồi đánh giá lại từ đầu.

## 5. Rủi ro & giới hạn thực tế

### 5.1. Data contamination và overfitting lên eval set

Data contamination xảy ra khi dữ liệu trong eval set vô tình lọt vào dữ liệu huấn luyện hoặc dữ liệu few-shot của mô hình khi đó điểm số đánh giá cao không phản ánh khả năng thực sự, mà chỉ là mô hình đã "nhìn thấy đáp án" từ trước. Rủi ro tương tự nhưng tinh vi hơn là overfitting lên eval set theo thời gian: khi một đội liên tục điều chỉnh prompt/hệ thống để tối đa hoá điểm số trên một eval set cố định, hệ thống dần "học" các đặc điểm riêng của chính bộ eval đó, thay vì cải thiện chất lượng tổng quát biểu hiện qua việc điểm nội bộ tăng đều nhưng phản hồi thực tế từ người dùng không cải thiện tương ứng. Biện pháp giảm thiểu: định kỳ làm mới một phần eval set bằng dữ liệu chưa từng dùng để tối ưu, và luôn đối chiếu điểm số nội bộ với tín hiệu chất lượng thực tế (phản hồi người dùng, tỷ lệ phải sửa tay).

## 5.2. Thiên lệch của LLM-as-judge

Dùng một mô hình ngôn ngữ để chấm điểm mô hình khác đã được ghi nhận có một số thiên lệch hệ thống (Zheng và cộng sự, 2023; Wang và cộng sự, 2023) đáng lưu ý khi diễn giải kết quả: xu hướng ưu ái câu trả lời dài hơn bất kể chất lượng nội dung (length bias), xu hướng ưu ái câu trả lời xuất hiện ở một vị trí cố định khi so sánh cặp (position bias), và có thể tự thiên vị cho câu trả lời do chính họ mô hình (hoặc cùng dòng mô hình) sinh ra (self-preference bias). Giảm thiểu bằng cách: hoán đổi vị trí các câu trả lời khi so sánh cặp để triệt tiêu position bias, chuẩn hoá độ dài hoặc yêu cầu giám khảo giải thích lý do chấm điểm thay vì chỉ cho điểm số, và đối chiếu định kỳ kết quả LLM-as-judge với một mẫu nhỏ được con người đánh giá độc lập để phát hiện sớm khi hai kết quả bắt đầu lệch nhau.

## 6. Checklist nhanh cho đội ngũ

- Eval set có phản ánh đúng phân bố câu hỏi thực tế, bao gồm cả các trường hợp lỗi đã biết (regression set) và trường hợp biên?
- Có tách riêng đánh giá theo từng thành phần hệ thống (truy xuất, sinh câu trả lời...) thay vì chỉ đo kết quả cuối cùng như hộp đen?
- Nếu dùng LLM-as-judge, đã hoán đổi vị trí khi so sánh cặp và đối chiếu định kỳ với đánh giá của con người chưa?
- Quy trình đánh giá có chạy tự động mỗi khi có thay đổi (prompt/RAG/fine-tune), so sánh với baseline, trước khi triển khai?
- Eval set có được làm mới định kỳ để tránh overfitting/data contamination theo thời gian?
- Điểm số nội bộ có được đối chiếu với tín hiệu chất lượng thực tế từ người dùng, không chỉ tin tưởng tuyệt đối vào con số?

## Đọc thêm

Tài liệu này tập trung vào các nguyên tắc và quy trình nền tảng của việc đánh giá mô hình. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến về xây dựng quy trình đánh giá trong bối cảnh ứng dụng production trên nền foundation model, cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

*"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.*

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

## Tài liệu tham khảo

- Zheng, L. et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." NeurIPS, 2023 (arXiv:2306.05685).
- Wang, P. et al. "Large Language Models are not Fair Evaluators." arXiv:2305.17926, 2023.
- Papineni, K. et al. "BLEU: a Method for Automatic Evaluation of Machine Translation." ACL, 2002.
- Lin, C-Y. "ROUGE: A Package for Automatic Evaluation of Summaries." ACL Workshop, 2004.
- Chang, Y. et al. "A Survey on Evaluation of Large Language Models." ACM TIST, 2024 (arXiv:2307.03109).
- OpenAI. "Evals framework for evaluating LLMs." [github.com/openai/evals](https://github.com/openai/evals), 2026.